ActPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF

Paper #XXX
Anonymous Submission

Abstract

AI coding agents require harnesses to apply behavioral policies for effective and safe operation: e.g. do not leak secrets, run tests before committing. Our empirical study of 64 projects shows that while project harness instructions are declared in natural language, 64% are behavioral directives.

Yet existing agent harnesses fail to connect intent-level declarations to deterministic system-level policies, leaving a *semantic gap* and making it difficult to ensure compliance or follow rules. While 83% of directives involve system-level behavior, prompt constraints operate at the probabilistic intent level. 81% of projects require cross-event state tracking, but tool-call guardrails lose track of system-level actions. 74% of system-level rules require intent-level project or task context to evaluate, while current OS-level mechanisms expect predefined static policy and return only system events without semantic context.

To address this, we argue the policy engine should be exposed as a programmable interface, allowing agents to dynamically define and adapt their policy for their own system-level actions based on their intent context. We realize this in ActPlane, which allows agents to declare cross-event behavior as labeled information-flow control (IFC) policies under a layered policy model, observing and enforcing the agents' actions across process, file, and network boundaries with semantic feedback. We implement ActPlane with eBPF and evaluate it across empirical policies, system workloads, and coding tasks, showing improved policy compliance with low end-to-end overhead.

CCS Concepts: • **Computer systems organization** → *Embedded and cyber-physical systems*; • **Security and privacy** → *Software and application security*; • **Software and its engineering** → *Software safety*.

Keywords: AI agents, eBPF, BPF-LSM, labeled information-flow control, information-flow control, provenance, semantic feedback

ACM Reference Format:
Paper #XXX. 2026. ActPlane: Letting Agents Harness Their Own System-Level Behavior with eBPF. In *Proceedings of 2026 ACM SIGOPS Annual Technical Conference (ATC '26).* ACM, New York, NY, USA, 11 pages.

1 Introduction

AI coding agents operate as long-running processes with access to shells, source trees, package managers, and external APIs [18]. They autonomously write code, run tests, and manage infrastructure across extended sessions.

As agents gain autonomy, they require *harnesses* that apply behavioral policies: constraints on what the agent may do, in what order, and under what conditions [5, 34]. Projects encode such policies in instruction files (`CLAUDE.md`, `AGENTS.md`) [8, 22].

These policies are pervasive and demand deterministic enforcement. Our empirical study of 64 projects (§3) finds that 64% of instruction-file statements are behavioral directives, and 83% of those involve system-level behavior: commands executed, files accessed, and network connections made. Because LLM compliance is inherently probabilistic [21], policy cannot reside inside the model alone: even cooperative agents routinely violate declared constraints through planning errors, shell-script side effects, and opaque tool behavior.

Yet existing harnesses fail to connect these intent-level declarations to deterministic system-level enforcement, leaving a *semantic gap*. Tool-call guardrails [31, 35] lose track of system-level actions when agents shell out or spawn subprocesses: the same `git push` can be reached through a tool call, a `bash -c` wrapper, or a compiled binary; and 81% of projects require cross-event state tracking that tool-level interception cannot provide. OS-level mechanisms [9, 20, 26] observe all system effects but expect pre-defined static policy and return only system events without semantic context; yet 74% of system-level directives require intent-level project or task context that only the agent possesses.

This gap arises because no existing layer lets the agent, the only entity with the context to instantiate its own rules, also program the OS-level mechanism that enforces them. Since only the agent itself knows the project and task context, it must author its own enforcement policy but must not enforce it: trusted to honestly declare intent, but not trusted to reliably follow it during execution. This separation calls for a programmable interface that lets agents dynamically define and adapt policy for their own system-level actions based on their intent context.

We realize this in ActPlane. Agents and developers declare behavioral policies in a compact domain-specific language (DSL) that agents themselves can generate from natural-language directives; ActPlane compiles them into labeled information-flow rules and loads them into an eBPF/BPF-LSM (Linux Security Modules) enforcement engine. Named

labels propagate across processes, files, and network endpoints at kernel hooks, encoding execution history as checkable state. When a labeled event matches a rule, ActPlane returns semantic feedback: which rule matched, why, and what compliant alternative satisfies it. Each rule independently selects one of three effects: notify (observe), block (pre-operation denial), or kill (post-operation termination). A layered authority model ensures that higher-authority rules (e.g., platform-level "do not leak secrets") cannot be weakened by agent-authored policy.

We make three contributions:

1. An **empirical study** of instruction files from 64 projects, showing that cross-event information-flow policies are pervasive (81% of repositories) and that 74% of system-level directives require project or task context that only the agent can supply (§3).
2. **ActPlane**, a programmable OS-level enforcement layer for agent harnesses. ActPlane compiles behavioral policies into eBPF/BPF-LSM labeled information-flow rules, enforces them across process, file, and network boundaries, and returns semantic feedback that reconnects rule matches to intent-level remediation (§4–5).
3. An **evaluation** across 607 directive-derived rules, system workloads, and a 20-task OctoBench subset. ActPlane improves policy compliance over prompt-only and tool-regex baselines while adding less than 2% end-to-end overhead on agent workloads (§6).

2 Background

Coding agents. AI coding agents such as Claude Code [1], Codex CLI [25], Cursor [2], and Devin [11] operate on repositories through an *execution harness* that mediates tool calls, shell commands, file access, network access, and model feedback [5, 34]. A single tool invocation can run arbitrary scripts, spawn subprocesses, and touch many files and endpoints before control returns to the model. Projects encode behavioral expectations in instruction files (`CLAUDE.md`, `AGENTS.md`) [8, 22], but the harness must decide how, and at what layer, to enforce them.

Information-flow control. Information-flow control (IFC) tracks how data or authority moves from sources to sinks over time. Static IFC systems embed flow constraints in type systems or language annotations [24]. Dynamic IFC and provenance systems attach labels to OS objects and propagate them at runtime: when a process reads a labeled file, the process acquires that label; when it forks, the child inherits it [10, 16, 27]. Later operations are checked against the accumulated label state. OS-level IFC systems such as Flume [20] and HiStar [37] enforce flow policies across processes and files in the kernel, while whole-system provenance frameworks such as CamFlow [27] and CamQuery [26] record and query cross-event label histories. IFC thus captures constraints whose correctness depends on execution history; this is exactly the pattern that agent workflow directives ("run tests before committing") exhibit.

Agent harness enforcement. Existing agent harnesses enforce policy at one of three levels. Prompt instructions rely on the model's own compliance, which is inherently probabilistic [21]. Tool-call guards such as AgentSpec [35] and Progent [31] intercept tool-call names and arguments deterministically, while application-level IFC systems such as FIDES [13] and CaMeL [14] track data flow within the agent loop. These mechanisms observe only the harness request, not the system-level effects that follow once a tool starts executing: a subprocess, a shell-out, or a compiled binary can bypass the tool boundary entirely. OS-level mechanisms (seccomp [12], AppArmor [6], BPF-LSM [33], Tetragon [9]) observe those effects but require statically pre-written policies and return opaque errors with no connection to the agent's declared directives. No current layer bridges both: connecting the agent's project context to deterministic OS-level observation and enforcement. The empirical study below asks whether real project instructions demand this bridge.

3 Empirical Study

We analyze real agent instruction files to answer a prerequisite question: do project instructions contain behavioral policies that demand OS-level, cross-event enforcement? This section characterizes the directives, classifies their enforcement requirements, and quantifies the context they need to become concrete rules.

3.1 Methodology

We collected public GitHub repositories containing `CLAUDE.md` or `AGENTS.md`, prioritizing the AI-agent ecosystem over filename-only matches and excluding non-code, inactive, fake-star, and stub repositories. The snapshot (2026-05-23 UTC) contains 64 repositories, 84 deduplicated instruction files, and 2,116 extracted statements. A *statement* is a coherent unit expressing one claim, directive, or constraint. A two-pass LLM-assisted pipeline extracted statements with source line ranges and four labels (content type, topic, enforcement level, context requirement); a validation script verified full source coverage and verbatim span matching. A stratified sample of 100 statements was independently reviewed by human annotators, with enforceability labels additionally cross-checked by independent Claude and Codex agents. The following subsections report results along the four labeling axes; E-RQ3 identifies the OS-enforceable subset used in the evaluation (§6). Table 1 collects representative statements that illustrate all four axes; the labels **S1**–**S8** are referenced throughout.

3.2 E-RQ1: Are Instruction Files Behavioral Policies?

What we test. Are instruction files primarily project documentation, or do they function as behavioral policies? A

Table 1. Representative corpus statements illustrating all four classification axes. Enforcement level and context requirement apply only to directives (— = not applicable).

	Statement	Type	Topic	Enforcement	Context
S1	"The backend uses Express with TypeScript."	Desc	Architecture	—	—
S2	"Always explain your reasoning before changes."	Dir	AI Integr.	Semantic	—
S3	"Prefer `const` over `let`."	Dir	Impl. Det.	Content	None
S4	"Never push to main directly."	Dir	Dev. Proc.	Per-event	None
S5	"Never modify upstream source code."	Dir	Dev. Proc.	Per-event	Project
S6	"Run the full test suite before committing."	Dir	Testing	Cross-event	Project
S7	"Data read from `.env` must not reach the network."	Dir	Security	Cross-event	Project
S8	"Do not update dependencies without approval."	Dir	Maint.	Per-event	Task

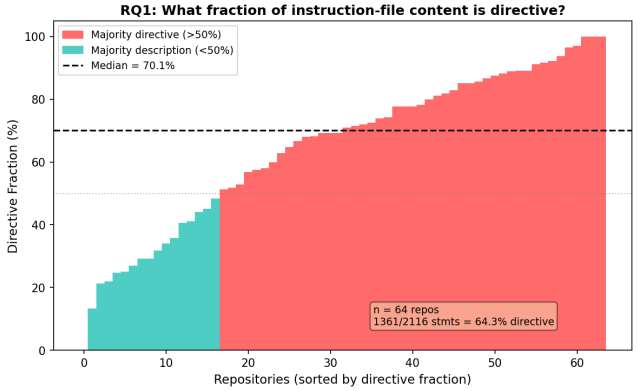

Figure 1. Directive fraction per repository by statement count. Most repositories contain a majority of directive statements.

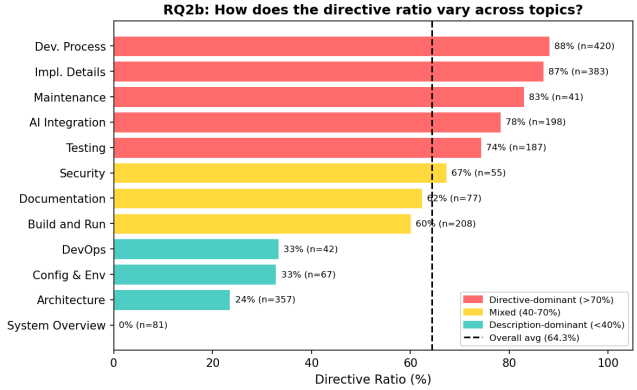

Figure 2. Directive ratio by topic. Development Process and Implementation Details are directive-dense; Architecture is mostly descriptive.

statement is a *directive* if it asks the agent to perform, avoid, or condition an action; otherwise it is descriptive context.

Finding. Instruction files are predominantly directive. Of the 2,116 statements, 1,361 (64.3%) are directives and 755 (35.7%) are descriptions. The median repository has 15 directives at a 71.0% directive fraction. This statement-level view diverges from line-count analysis: directives tend to be terse, while descriptions include long directory listings and architecture notes. A file that looks balanced by lines can still contain many independent directives the agent must follow.

3.3 E-RQ2: Which Topics Contain Directives?

What we test. Does the directive fraction vary by topic? Each statement is assigned to one of 12 topic categories adapted from prior instruction-file studies, applied at statement granularity rather than file granularity.

Finding. Directive density varies sharply across topics. Development Process and Implementation Details are directive-heavy (86.7% and 85.2%, respectively). Architecture is mostly descriptive: directory layouts and design summaries make it one of the largest topics by line count, yet only 23.0% of its statements are directives. File-level topic labels can therefore be misleading for enforcement: a file classified as "Architecture" may mostly describe the project, while a shorter Development Process section packs many enforceable rules.

3.4 E-RQ3: Which Directives Require OS-Level Enforcement?

What we test. Which directives have system-observable counterparts, and at what enforcement layer? Each directive is classified into the first matching tier of an enforcement waterfall: *semantic-only* (reasoning, communication, or output style), *content* (predicates over file contents), *per-event* (a single command, file access, or network connection), and *cross-event* (history, ordering, lineage, or information flow). We call the union of content, per-event, and cross-event directives *system-observable*; the per-event and cross-event subset that ActPlane can enforce at OS hooks is *OS-enforceable*.

Finding. The majority of directives are system-observable. Of 1,361 directives, 1,127 (82.8%) involve commands, files, network effects, or file contents. Breaking these down: 234 (17.2%) are semantic-only, 520 (38.2%) require content inspection, 392 (28.8%) match a single OS event, and 215 (15.8%) require cross-event state. The 607 per-event and cross-event

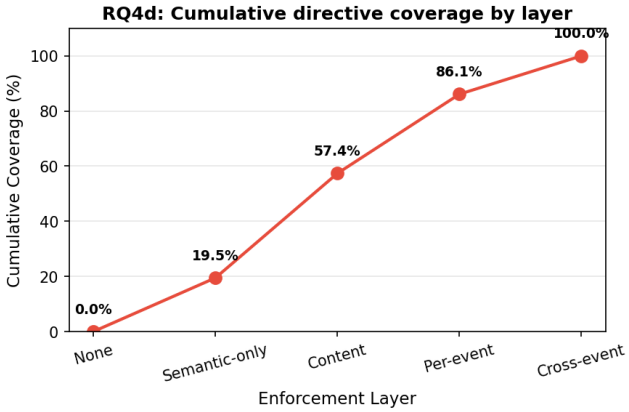

Figure 3. Cumulative directive coverage by enforcement layer. Linters cover 55.4%; per-event matching reaches 84.2%; cross-event IFC closes the remaining 15.8%.

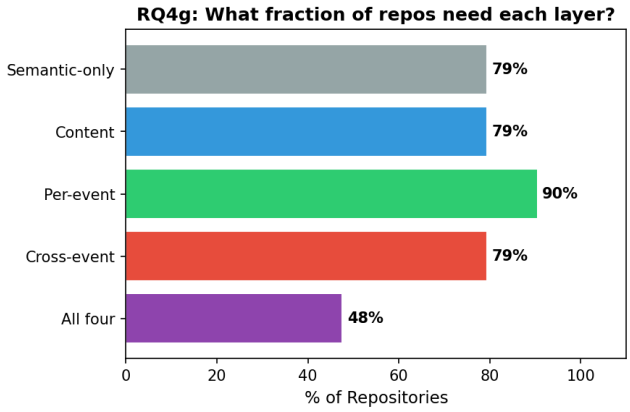

Figure 4. Fraction of repositories requiring each enforcement layer. 81% need cross-event tracking; 43% need all four layers.

directives form the OS-enforceable subset that ActPlane targets (Table 1, S2–S7, illustrates all four levels). Figure 3 shows the cumulative view: content inspection alone covers 55.4% of directives; adding per-event matching reaches 84.2%; the remaining 15.8% require cross-event IFC. At the repository level, cross-event policies are pervasive: 81% of repositories contain at least one, and 43% require all four enforcement layers (Figure 4).

3.5 E-RQ4: What Context Is Needed to Instantiate Rules?

What we test. Can system-observable directives be instantiated as concrete rules from their text alone, or do they require additional context? A directive is *self-contained* if all commands, paths, and patterns are explicit. It requires *project context* if it references an unresolved repository-specific concept ("the full test suite," "the migration tool"). It requires *task context* if it depends on the current user request or a per-session grant ("unless explicitly requested," "without approval").

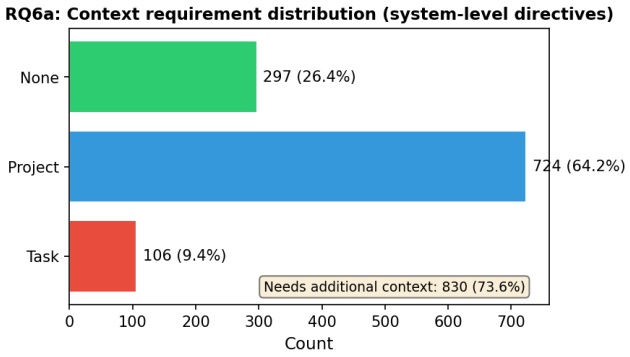

Figure 5. Context required to instantiate system-observable directives. 73.6% require project or task context beyond the directive text.

Finding. Most system-observable directives are not self-contained. Of the 1,127 system-observable directives, only 26.4% specify all commands, paths, and patterns needed to produce a concrete rule directly. Another 64.2% require *project context*: the directive mentions "the test suite," "the linter," or "upstream source," whose concrete command or path must be resolved from the repository. The remaining 9.4% require *task context*: the rule depends on the current user request or a per-session grant (Figure 5). This quantifies the gap between a natural-language instruction and a deployable enforcement rule: statically authored rules can cover only the self-contained fraction, while the majority require an entity with project and task context, such as the agent itself, to read the repository, interpret the current task, and produce a concrete policy before enforcement begins (Table 1: S4 is self-contained; S5–S7 require project context; S8 requires task context).

4 Design

The empirical study establishes one overarching requirement: connect the agent's intent-level context to deterministic OS-level enforcement. Achieving this raises three design challenges.

C1: Agent-authorable OS-level policy (§4.3). 74% of system-observable directives require project or task context that only the agent possesses, so the agent must author its own enforcement rules. Yet OS-level enforcement demands concrete, deterministic predicates. The policy language must be simple enough for an LLM to generate correctly, yet compile to efficient kernel-level matching.

C2: Cross-event state at syscall speed (§4.4). 81% of repositories require policies that span multiple operations: "run tests before committing" depends on whether a test execution occurred after the last source change. Maintaining

this execution history across processes, files, and network endpoints at the kernel level must not impose prohibitive per-syscall overhead.

C3: Dynamic policy under authority constraints (§4.5). Agents must be able to adapt policy at runtime: a sub-agent may need a narrower scope, or the current task may require an exception to a standing rule. Policy evolution must be monotonically tightening: agents and sub-agents may add rules, specialize targets, or narrow scope, but must never weaken platform or project constraints.

4.1 Threat Model and Assumptions

ActPlane targets a *cooperative but unreliable* agent. The agent's original intent is honest: it aims to follow project constraints. But execution may diverge due to planning errors, shell-script side effects, opaque tool behavior, or prompt injection [28] that causes the agent to act against its declared policy. Higher-authority policies (platform, project) are authored by trusted principals and cannot be weakened by the agent; they guard against both unreliable execution and hijacked agent-level declarations. The system does not target a malicious process that attacks the kernel, exploits the loader, or deliberately evades policy through adversarial obfuscation.

The trusted computing base (TCB) consists of the compiler, the eBPF enforcement engine, the runner, and higher-authority policies. The agent's entire process tree, including tools, shells, subprocesses, and sub-agents, constitutes the monitored domain. Kernel compromise, root or `CAP_BPF` compromise, deliberate side channels, and semantic content checks not observable at syscall boundaries are out of scope.

4.2 System Overview

ActPlane addresses each challenge with one abstraction: a policy DSL (C1) that compiles agent-authored directives into kernel-level IFC tables; a labeled information-flow model (C2) that propagates cross-event state across OS objects at object granularity; and a layered authority model (C3) that allows runtime policy adaptation while ensuring monotonic tightening. When a rule match occurs, ActPlane returns semantic feedback to the agent: the rule name, reason, and remediation.

4.3 Policy DSL

The DSL mediates between intent-level instructions and system-level enforcement. A policy contains *sources* that introduce labels, *rules* that match labeled events at sinks, and optional *gates* that express ordering or controlled declassification. The DSL restricts expressiveness to source/sink/gate patterns that compile to fixed-size rodata, enabling bounded verification cost and agent-authorability.

Sources bind labels to system objects: an `exec` source labels processes that execute a matching binary; a `file` source labels matching paths; a network source labels matching endpoints. Rules bind an effect (`notify`, `block`, or `kill`) to an operation (`exec`, `read`, `write`, `unlink`, or `connect`). Conditions express common agent workflow constraints: a target allow-list, a lineage gate, or a temporal `after ... since ...` ordering gate.

```
source AGENT = exec "claude"
source SECRET = file "**/.env"

# Per-event: from CoplayDev/unity-mcp
rule no-push-main:
  kill exec "git" "push" "main" if AGENT
  because "Do not push to main; open a PR instead."

# Cross-event: from chenhg5/cc-connect
rule tests-before-commit:
  block exec "git" "commit"
   if AGENT unless after exec "go" since write "**/*.go"
  because "Run `go test ./...` before committing."
```

Figure 6. Two ActPlane rules drawn from real projects. The first is a per-event rule; the second is a cross-event rule that requires temporal state.

Figure 6 illustrates the two main rule shapes. The first is per-event: if the agent process pushes to `main`, the event matches immediately. The second is cross-event: committing is compliant only if the required test command ran after the most recent source write. This distinction motivates the DSL's two predicate classes: per-event matches and cross-event gates.

The DSL is intentionally narrower than a general mandatory access-control language. It targets the policy shapes that recur in agent instruction files: prohibit a command, restrict writes to a scope, block a sink after a sensitive read, require a validation step before a release action, and route sensitive data through an approved tool. The compiler lowers these patterns into fixed-size kernel tables; policy authors need not reason about BPF maps, verifier constraints, or binary layout.

4.4 Labeled Information-Flow Model

ActPlane represents policy state as labels on OS objects. Processes, files, and network endpoints each carry a 64-bit label mask, enabling cross-boundary flow policies (e.g., "data read from `.env` must not reach a network endpoint"). A source declaration adds labels when an object matches the source pattern. Let $L(x)$ denote the label mask of object x. Labels propagate through observed system-level edges:

- **fork**$(p \to c)$: $L(c) := L(c) \cup L(p)$.
- **exec**(p, f): $L(p) := L(p) \cup L(f) \cup S(f)$, where $S(f)$ is the set of source labels matching f.
- **read**(p, f): $L(p) := L(p) \cup L(f)$.
- **write**(p, f): $L(f) := L(f) \cup L(p)$.

- **connect**(p, e): $L(e) := L(e) \cup L(p)$.

These transfer functions maintain a key invariant: if an object carries label ℓ, then information from a source tagged with ℓ has reached that object through observed execution edges. Rules check whether an event's subject or target carries required labels, carries forbidden labels, or satisfies a temporal gate.

The model operates at object granularity rather than byte granularity, trading precision for low-overhead whole-session coverage. This is a deliberate fit for agent harnesses: repository instructions refer to commands, files, directories, endpoints, and workflow steps, not individual bytes. Object-level labels capture cross-event compliance patterns that tool-call filters miss, without incurring the complexity of byte-level taint tracking [19].

Labels are monotonic by default: propagation adds labels but does not remove them. When a policy requires controlled release, the DSL names an explicit gate, such as a redaction command or review tool, that clears specified labels for subsequent events. Declassification is thus a declared policy action, not an implicit side effect of the agent's execution.

4.5 Layered Policy Authority

Agents must adapt policy at runtime: a sub-agent spawned for a narrow sub-task should operate under tighter constraints, and the current task may require an exception to a standing rule; but adaptation must never weaken platform or project constraints.

ActPlane organizes policy as a layered authority stack: platform/operator, project-maintainer, and agent/session. Policy evolution is *monotonically tightening*: each layer may add rules, specialize targets, introduce labels, or narrow scope, but no layer may delete, rewrite, declassify, or weaken rules from a higher-authority layer. When an agent forks a sub-agent, the child inherits the parent's label set and effective rules; the sub-agent may add further restrictions but cannot shed inherited constraints.

Higher-authority rules that admit exceptions use the same gate mechanism as cross-event rules: the rule names an explicit action the agent must perform to satisfy the condition. For example, a project rule "do not force-push unless necessary" can require the agent to write a declaration file (e.g., `.actplane/force-push-necessary`) before the push is allowed. The agent satisfies the gate by creating the file, which the IFC engine tracks like any other label-propagating event. Higher-authority policy can further constrain which processes may write the declaration file, keeping exception paths under platform or project authority.

The prototype resolves the initial authority stack at compile or load time. Runtime policy additions (sub-agent restrictions, task-specific rules) are loaded as additive overlays; the kernel merges them with the existing rule tables. The kernel does not parse configuration or resolve project context on the event path; agents supply context and policy structure, while the OS engine enforces the compiled result uniformly across tools and subprocesses.

5 Implementation

ActPlane consists of a Rust compiler/runner (3.2 K LoC) and an eBPF enforcement engine (1.8 K LoC of BPF C). The compiler parses the DSL, allocates label bits, lowers Boolean label expressions and path/network patterns, and emits a fixed-size `#[repr(C)]` configuration blob consumed directly by the eBPF loader. Human-readable rule names and reasons stay in userspace metadata; the kernel receives only compact rule tables and emits rule identifiers when events match.

The eBPF engine attaches to process, file, and network hooks via BPF-LSM (pre-operation enforcement) and tracepoints (observation and post-operation termination). It maintains label state in per-object BPF maps, applies the transfer functions from §4, and evaluates the compiled rules on each observed event. Userspace does not re-detect violations: it loads the policy, starts the target process, and formats kernel-reported matches into semantic feedback.

The runner (`actplane run -- <cmd>`) discovers the project policy file, compiles and loads it before the target starts, seeds the target process tree with the agent label, and records kernel matches in a feedback file. Compatible agent hooks relay this feedback into the model's next turn. The runner is intentionally thin: the policy decision stays in the kernel, while the runner only connects policy, execution, and agent-facing diagnostics.

6 Evaluation

We evaluate ActPlane along three axes. RQ1 measures end-to-end policy compliance on directive-derived rules from the empirical corpus; RQ2 quantifies per-event and end-to-end overhead; RQ3 tests repository-grounded coding tasks on an OctoBench subset with the official judge.

RQ1 (Policy compliance) Compared with prompt-only, tool-layer, and feedback-ablation baselines, does ActPlane improve policy compliance for directive-derived rules across direct and bypass execution paths?

RQ2 (Overhead) What is the per-event and end-to-end overhead of ActPlane's label propagation and rule checking?

RQ3 (Repository-grounded feedback) On a scaffold-aware repository-grounded coding benchmark, does ActPlane's OS-level enforcement and semantic feedback improve official policy compliance?

6.1 Experimental Setup

Hardware and kernel. All experiments run on a single machine with an Intel Core Ultra 9 285K CPU (24 hardware cores), 125 GiB RAM, Linux 6.15.11, ext4 filesystem, and libbpf 1.x with `bpf_loop` support. The live enforcement runs

require BPF-LSM active (`bpf` in `/sys/kernel/security/lsm`); the RQ2 overhead artifacts record the exact LSM state used for each performance run.

Agent environment. The active agent harnesses are Claude Code (via the ActPlane feedback hook) and OpenAI Codex CLI (via tool-error feedback). Exact versions are recorded in per-run metadata.

Baseline systems. All baselines use the *same* DSL rules; only the enforcement layer differs. Tool-layer baselines are Python scripts that read the same `rule.yaml` and implement DSL semantics at the tool-call level. Kernel baselines are ActPlane with features selectively disabled. We compare seven configurations: prompt-only, TL-1 (single-event tool guard), TL-N (multi-event tool guard), app-level IFC, per-event eBPF, kernel IFC without feedback, and full ActPlane. The tool-layer and app-level baselines represent AgentSpec/Progent-style guards [31, 35] and FIDES/CaMeL-style tool-call label tracking [13, 14]; the kernel baselines isolate the contributions of cross-event state and semantic feedback.

6.2 Rule Set Construction

The empirical study identifies 607 OS-enforceable directives: 392 per-event and 215 cross-event. RQ1 uses this subset because these directives can be represented at process, file, or network hooks. Semantic-only and content directives are excluded: the former have no system-observable counterpart; the latter require file-content inspection rather than OS-level IFC. The evaluation pipeline enforces strict separation of concerns across four actors.

Pipeline overview. The pipeline has five stages with strict actor separation. A translation LLM reads the sampled directive text, the project's `CLAUDE.md`, the cloned repository, and the DSL reference to produce a rule file; it cannot see traces or ground truth. A separate trace generator reads the directive and project structure to produce violation and compliant trace files, each with a ground-truth header; it cannot see the rule file, preventing circular bias. A deterministic script then wraps the triggering command from each direct trace in `bash -c`, subprocess, and compiled-binary variants, with no LLM involvement. The eval harness replays each trace under each system, collects the system response, and issues one API call to a tested LLM, a smaller model standing in for the agent, for the next-action decision. Finally, a scorer compares the tested LLM's decision against ground truth and classifies the outcome as TP, TN, FP, or FN. Because the translation LLM and the trace generator are different actors, translation blind spots do not hide in traces; because the tested LLM never sees ground truth, it acts like a real agent. All systems run on the *same* agent-generated rules, so translation errors are shared noise and differences between systems reflect architectural capability. Figure 6 shows representative per-event and cross-event rules.

Table 2. RQ1 confusion matrix by guardrail system.

System	DCR	TP	TN	FP	FN	Judged
Prompt-filter	57.5%	50	81	33	64	228
Tool-regex	59.2%	50	85	29	64	228
ActPlane-opaque	67.5%	42	112	2	72	228
ActPlane	**77.6%**	**79**	98	16	**35**	228

6.3 RQ1: End-to-End Policy Compliance

Method. RQ1 uses a trace-conditioned benchmark derived from the OS-enforceable directive corpus. The current paper-facing sample contains 38 statements. For each statement we construct six manifest-listed traces: three benign traces (`canonical_compliant`, `allowed_effect_compliant`, and `lookalike_compliant`) and three violation traces (`visible_violation`, `script_visible_violation`, and `opaque_fixture_violation`). This yields 228 traces and 912 system–trace cells across four systems: prompt-filter, tool-regex, ActPlane without structured feedback (ActPlane-opaque), and full ActPlane.

Each trace is a JSONL transcript with a ground-truth header followed by a user prompt and concrete tool calls that establish the repo state. The tested agent then continues for a bounded number of real Agent SDK tool steps. All systems use the same trace, model, Docker copy-on-write repo execution, and trajectory judge. The prompt-filter baseline applies a stateless LLM classifier to proposed tool actions using the natural-language directive; tool-regex applies the per-case raw tool-input regex policy; and ActPlane runs the generated OS policy below the tool layer through the eBPF/ActPlane runner. Ground truth and policy artifacts are not shown to the tested agent.

A separate LLM trajectory judge assigns TP, TN, FP, or FN from the raw runner result. TP means a valid violation trace received guardrail-visible intervention or recovery; TN means a valid benign trace completed without wrongful guardrail intervention; FP means a benign trace was wrongly blocked, reported, failed, or steered away; FN means a violation trace was missed or left unrecovered. We report directive compliance rate $\mathrm{DCR} = (\mathrm{TP}+\mathrm{TN})/(\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN})$. For this snapshot, each (*system*, *statement*, *trace*) cell uses the latest judged result matching the current trace artifact; all 912 cells are present and none are stale relative to the trace files.

Results. ActPlane improves overall compliance by 18.4 percentage points over tool-regex and 20.1 points over prompt-filter. The improvement comes from two complementary effects. First, ActPlane observes runtime side effects below the tool layer, so it can detect subprocess and fixture-mediated violations that are not available to prompt or raw tool-input filters. Second, it reduces over-blocking on benign near-boundary traces: full ActPlane has 16 FPs, compared with 33 for prompt-filter and 29 for tool-regex.

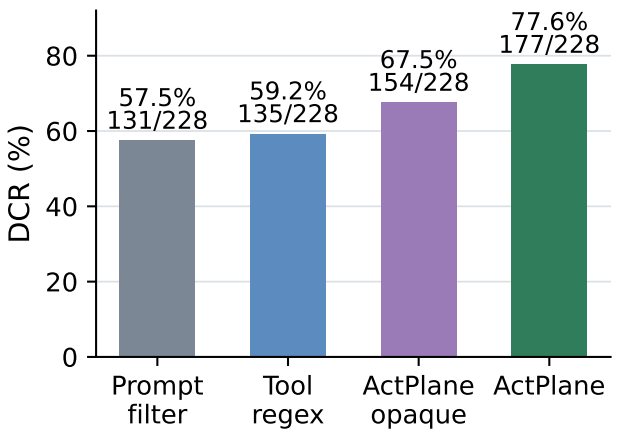

Figure 7. Overall RQ1 directive compliance rate across 228 traces.

The feedback ablation isolates the value of semantic recovery information. ActPlane-opaque has very few false positives, but it misses many violations (72 FNs) because the agent receives no structured corrective payload after enforcement. Full ActPlane cuts those FNs roughly in half while preserving substantially lower FP counts than the prompt and regex baselines. Thus, the result is not simply that kernel enforcement can stop more effects; model-facing feedback is part of the end-to-end guardrail.

6.4 System Comparison Across Execution Paths

This sub-experiment isolates three architectural failure modes: tool-layer guards losing coverage on bypass paths, OS mechanisms lacking cross-event state, and kernel mechanisms returning bare errors without semantic context. We compare all seven systems on the same directives.

Method. We select all rules from the **TP set** of RQ1 (rules where ActPlane produced the correct outcome on direct traces). This isolates translation noise, so any difference between systems is purely architectural. For each TP rule, we take the RQ1 violation trace and programmatically generate **3 bypass variants** by replacing the triggering tool call with an indirect execution path:

`bash -c` The triggering command is wrapped in `bash -c '<cmd>'`.
Python subprocess The command is wrapped in `python3 -c "import subprocess; subprocess.run([...])"`.
Compiled binary The command is embedded in a pre-built helper binary (`./commit-helper`).

Each TP rule yields 4 traces (1 direct + 3 bypass). Each trace uses the same JSONL format as RQ1 and the same minimal agent harness: deterministic replay → system responds → LLM handles the response (1–2 API calls per trace).

For kernel systems (ActPlane, Kernel IFC, Per-event eBPF) the trace runs under `sudo actplane run [-flags]`. For tool-layer and app-level baselines the replay executor intercepts tool calls through the Python baseline checker.

Table 3. Directive compliance rate by system and execution path.

System	Direct	Bypass	Gap
ActPlane	TODO	TODO	TODO
Kernel IFC	TODO	TODO	TODO
Per-event eBPF	TODO	TODO	TODO
App-level IFC	TODO	TODO	TODO
TL-N	TODO	TODO	TODO
TL-1	TODO	TODO	TODO

Table 4. Directive compliance rate by system and enforcement level.

System	Per-event	Cross-event	Total
ActPlane	TODO/N_1	TODO/N_2	TODO
Kernel IFC	TODO/N_1	TODO/N_2	TODO
Per-event eBPF	TODO/N_1	TODO/N_2	TODO
App-level IFC	TODO/N_1	TODO/N_2	TODO
TL-N	TODO/N_1	TODO/N_2	TODO
TL-1	TODO/N_1	TODO/N_2	TODO

Metrics.

- **Directive compliance rate** per system, broken down by enforcement level (per-event vs. cross-event) and execution path (direct vs. bypass).
- **Bypass gap**: the difference between direct and bypass compliance per system; this shows which systems lose coverage on indirect paths.
- **Feedback recovery gap**: Kernel IFC (bare `-EPERM`) vs. ActPlane (semantic feedback); this shows the value of feedback for agent recovery.

Kernel IFC and ActPlane share identical detection; the compliance difference isolates the value of semantic feedback for agent recovery.

6.5 RQ2: Overhead

6.5.1 Macrobenchmarks (End-to-End Workloads). We measure end-to-end overhead on two deterministic workloads under no-hit ActPlane configurations. The agent trace suite replays 20 corpus-derived traces (68 tool actions, 20 Bash subprocesses). The Linux kernel build compiles `defconfig` + `vmlinux` with `make -j24` in a clean output directory. Each workload runs three trials per configuration. Fixed workloads eliminate LLM inference variance and isolate ActPlane's runtime cost.

6.5.2 Microbenchmarks (Per-Syscall Latency). We measure ActPlane's per-event latency for five syscall types (`fork`,

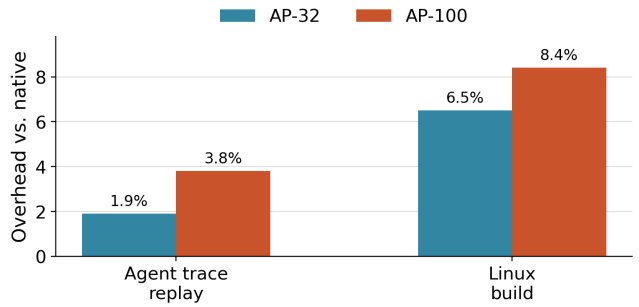

Figure 8. End-to-end overhead on deterministic workloads, normalized to native execution. At 32 active no-hit rules, ActPlane adds 1.9% on agent-trace replay and 6.5% on a Linux build; at 100 rules, overhead remains below 8.4%.

Table 5. Per-operation latency in microseconds for no-hit configurations. Values are medians across seven trials.

Operation	Native p50	AP-32 p50	AP-100 p50	AP-100 p99
`open`	0.58	6.92	13.40	15.36
`write`	0.27	0.79	0.84	1.59
`connect`	0.58	1.98	3.17	3.90
`fork`	48.94	74.05	69.33	178.01
`exec`	248.30	314.86	317.03	490.37

`exec`, `open`, `write`, `connect`) under five no-hit configurations: native, and ActPlane with 1, 10, 32, and 100 active rules (AP-N denotes ActPlane with N rules). Each configuration × syscall type runs 10K–100K iterations pinned to a single CPU core. For each trial we compute p50 and p99 latencies; Table 5 reports the median across seven trials.

Measured operations. `fork`: `fork()` + parent `waitpid()`. `exec`: `fork()` + child `execve("/bin/true")` + parent `waitpid()`. `open`: `open(path, O_RDONLY|O_CLOEXEC)` with `close()` outside the timed region. `write`: `write(fd, 4096)` to an already-open temp file. `connect`: UDP `connect(127.0.0.1:9)` on an already-created socket.

Results. The largest per-call cost is on `open`: AP-100 adds 12.8 µs at p50 (23× native), because each open triggers a path hash (FNV-1a), a file-label map lookup, and a linear scan of up to 100 rules. `connect` adds 2.6 µs and `write` 0.57 µs at p50; both involve shorter matching paths. `fork` and `exec` include scheduler and process-management overhead, so their tail latencies are noisier. Despite the high per-call multiplier on `open`, the macrobenchmark impact remains low (1.9%–8.4%) because the absolute added cost (12.8 µs) is small relative to the computation, I/O, and scheduling time between successive opens. The per-call overhead therefore does not accumulate into a significant fraction of end-to-end wall-clock time.

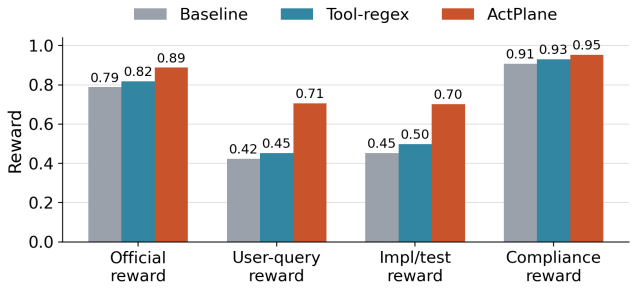

Figure 9. OctoBench 20-task selected subset. ActPlane improves official policy compliance over both baseline and tool-regex.

All overhead measurements use no-hit configurations, which exercise steady-state label propagation and rule scanning without violation reporting or userspace feedback formatting.

6.6 RQ3: Repository-Grounded Policy Compliance (OctoBench)

Goal. Unlike RQ1, which isolates single decision points, RQ3 evaluates ActPlane on complete coding-agent tasks in real repository environments, scored by the official OctoBench whole-case checklist judge.

Benchmark and subset. OctoBench [15] is a scaffold-aware repository-grounded benchmark with 217 Dockerized tasks spanning system prompts, tool schemas, project files (`CLAUDE.md`, `AGENTS.md`), and user queries. We use the official evaluator unchanged. Because ActPlane cannot observe purely semantic checks (e.g., tone), we select a 20-task subset with system-observable policy points and case-specific policies. Each task runs under three conditions: baseline (no enforcement), tool-regex (case-specific tool-call hooks), and ActPlane (OS-level hooks plus semantic feedback).

Metrics. The primary metric is official OctoBench reward (fraction of checklist policies judged satisfied). We additionally report three diagnostic submetrics from the same checklist results: *user-query reward* (user-task checks only), *implementation/test reward* (implementation, testing, configuration, and modification checks), and *compliance reward* (compliance-typed checks). All submetrics use the official LLM-based checklist judge.

Results. On the 20-task subset, official reward is 0.788 (baseline), 0.818 (tool-regex), and 0.888 (ActPlane): a 10.0-point gain over baseline and 7.0 points over tool-regex. The improvement extends beyond generic compliance: user-query reward rises by 28.2 points and implementation/test reward by 25.0 points over baseline. These results support the claim that OS-level enforcement with semantic feedback improves policy compliance in repository-grounded coding agents. We

note that OctoBench uses an LLM checklist judge rather than deterministic test scripts, so we do not claim deterministic task completion.

7 Related Work

In-kernel provenance and information flow. Whole-system provenance systems such as PASS [23], Hi-Fi [29], LPM [4], and CamFlow [27] record audit-grade provenance graphs via LSM hooks but do not compile agent-oriented policy or return semantic feedback. CamQuery [26] is the closest prior system: it evaluates provenance queries inline in the kernel, propagates labels across OS objects, and can deny matching operations. ActPlane shares this label-propagation model but targets cooperative AI agents rather than adversarial intrusions, compiles an agent-facing source/sink DSL rather than general provenance queries, and returns semantic feedback to the matching actor. Dynamic taint-tracking systems [10, 16, 19, 36] operate at byte or instruction granularity for vulnerability detection; ActPlane operates at object granularity, trading precision for low-overhead whole-session coverage.

eBPF enforcement and agent guardrails. BPF-LSM [32, 33] lets eBPF programs attach to security hooks and return allow/deny verdicts; ActPlane uses it as its pre-operation enforcement backend. Contemporary eBPF tools such as Tetragon, Tracee, and eBPF-PATROL [3, 9, 17] provide per-event predicates or single-channel lineage flags; ActPlane propagates multi-label information flow across channels and checks cross-event conditions. At the agent layer, AgentSpec and Progent [31, 35] enforce deterministic guards at the tool-call boundary; ActPlane complements them below the tool API, where shell-outs and subprocesses remain visible. FIDES and CaMeL [13, 14] apply typed IFC within the agent loop but do not expose a layered authority stack that separates platform, project, and agent policy; Flume [20] provides decentralized labels with explicit capabilities but requires per-application integration rather than a compile-time authority hierarchy. ActPlane applies information-flow reasoning at the OS boundary with a layered authority model, where observation persists across context windows and cannot be circumvented by subprocess indirection.

Agent instruction files. Recent studies characterize `CLAUDE.md` and `AGENTS.md` files along dimensions of content taxonomy, maintenance practices, and task efficiency [7, 8, 22, 30]. These works classify at file or section-heading granularity. Our empirical study (§3) classifies at statement level along four axes, specifically asking which instructions require cross-event enforcement and which need project or task context to instantiate.

8 Conclusion

An empirical study of 64 projects shows that agent instruction files are behavioral policies whose enforcement demands OS-level observation, cross-event state, and agent-provided context. ActPlane addresses this by letting agents author labeled information-flow policies that an eBPF/BPF-LSM engine enforces below the tool layer, returning semantic feedback when rules match. A layered authority model prevents agents from weakening higher-authority constraints. ActPlane does not cover semantic-only directives (17%) or content-inspection directives (38%) that require a linting layer, and the cooperative threat model excludes adversarial evasion. Extending coverage to content-level checks and multi-agent domains are natural next steps.

References

[1] Anthropic. 2025. Claude Code. https://docs.anthropic.com/en/docs/claude-code.

[2] Anysphere. 2025. Cursor: The AI Code Editor. https://cursor.com/.

[3] Aqua Security. 2026. Tracee: Linux Runtime Security and Forensics using eBPF. https://github.com/aquasecurity/tracee.

[4] Adam Bates, Dave Tian, Kevin R. B. Butler, and Thomas Moyer. 2015. Trustworthy Whole-System Provenance for the Linux Kernel. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, DC, 319–334. https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/bates

[5] Birgitta Boeckeler. 2026. Harness Engineering for Coding Agent Users. https://martinfowler.com/articles/harness-engineering.html.

[6] Canonical Ltd. 2024. AppArmor Security Profiles. https://apparmor.net/.

[7] Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. 2025. Agent READMEs: An Empirical Study of Context Files for Agentic Coding. arXiv:2511.12884. https://arxiv.org/abs/2511.12884

[8] Worawalan Chatlatanagulchai, Kundjanasith Thonglek, Brittany Reid, Yutaro Kashiwa, Pattara Leelaprute, Arnon Rungsawang, Bundit Manaskasemsak, and Hajimu Iida. 2025. On the Use of Agentic Coding Manifests: An Empirical Study of Claude Code. arXiv:2509.14744. https://arxiv.org/abs/2509.14744

[9] Cilium Project. 2026. Tetragon: eBPF-based Security Observability and Runtime Enforcement. https://tetragon.io/.

[10] James Clause, Wanchun Li, and Alessandro Orso. 2007. Dytan: A Generic Dynamic Taint Analysis Framework. In *Proceedings of the 2007 International Symposium on Software Testing and Analysis*. Association for Computing Machinery, London, United Kingdom, 196–206. doi:10.1145/1273463.1273490

[11] Cognition. 2025. Devin: The First AI Software Engineer. https://devin.ai/.

[12] Jonathan Corbet. 2015. A seccomp overview. https://lwn.net/Articles/656307/.

[13] Manuel Costa, Boris Koepf, Aashish Kolluri, Andrew Paverd, Mark Russinovich, Ahmed Salem, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2025. Securing AI Agents with Information-Flow Control. arXiv:2505.23643. https://arxiv.org/abs/2505.23643

[14] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. 2025. Defeating Prompt Injections by Design. arXiv:2503.18813. https://arxiv.org/abs/2503.18813

[15] Yangruibo Ding, Siyuan Cheng, Xiaohan Wang, Yifan Song, Yiming Wang, Markus Mock, and Chenguang Wang. 2026. OctoBench: Benchmarking Scaffold-Aware Instruction Following in Repository-Grounded Agentic Coding. arXiv:2601.10343. https://arxiv.org/abs/2601.10343

[16] William Enck, Peter Gilbert, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. 2010. TaintDroid: An Information-Flow Tracking System for Realtime Privacy Monitoring on Smartphones. In *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. USENIX Association, Vancouver, Canada, 393–407. https://www.usenix.org/conference/osdi10/taintdroid-information-flow-tracking-system-realtime-privacy-monitoring

[17] Sangam Ghimire, Nirjal Bhurtel, Roshan Sahani, and Sudan Jha. 2025. eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments. arXiv:2511.18155. https://arxiv.org/abs/2511.18155

[18] Yuxuan Jiang, Meng Xu, Yiwen Song, and Xinwen Fu. 2025. AgentSight: Bridging the Semantic Gap Between Agent Intent and Low-Level Actions. arXiv:2508.02736. https://arxiv.org/abs/2508.02736

[19] Vasileios P. Kemerlis, Georgios Portokalidis, Kangkook Jee, and Angelos D. Keromytis. 2012. libdft: Practical Dynamic Data Flow Tracking for Commodity Systems. In *Proceedings of the 8th ACM SIGPLAN/SIGOPS Conference on Virtual Execution Environments*. Association for Computing Machinery, London, United Kingdom, 121–132. doi:10.1145/2151024.2151042

[20] Maxwell Krohn, Alexander Yip, Micah Brodsky, Natan Cliffer, M. Frans Kaashoek, Eddie Kohler, and Robert Morris. 2007. Information Flow Control for Standard OS Abstractions. In *Proceedings of the 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP '07)*. ACM, 321–334.

[21] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. In *Transactions of the Association for Computational Linguistics*, Vol. 12. 157–173.

[22] Jai Lal Lulla, Seyedmoein Mohsenimofidi, Matthias Galster, Jie M. Zhang, Sebastian Baltes, and Christoph Treude. 2026. On the Impact of AGENTS.md Files on the Efficiency of AI Coding Agents. arXiv:2601.20404. https://arxiv.org/abs/2601.20404

[23] Kiran-Kumar Muniswamy-Reddy, David A. Holland, Uri Braun, and Margo Seltzer. 2006. Provenance-Aware Storage Systems. In *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. USENIX Association, Boston, MA, 43–56. https://www.usenix.org/conference/2006-usenix-annual-technical-conference/provenance-aware-storage-systems

[24] Andrew C. Myers. 1999. JFlow: Practical Mostly-Static Information Flow Control. In *Proceedings of the 26th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '99)*. ACM, 228–241.

[25] OpenAI. 2025. Codex CLI. https://github.com/openai/codex.

[26] Thomas F. J.-M. Pasquier, Xueyuan Han, Thomas Moyer, Adam Bates, Olivier Hermant, David Eyers, Jean Bacon, and Margo Seltzer. 2018. Runtime Analysis of Whole-System Provenance. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, Toronto, Canada, 1601–1616. doi:10.1145/3243734.3243776

[27] Thomas F. J.-M. Pasquier, Jatinder Singh, David Eyers, and Jean Bacon. 2017. CamFlow: Managed Data-Sharing for Cloud Services. *IEEE Transactions on Cloud Computing* 5, 3 (2017), 472–484. doi:10.1109/TCC.2015.2489211

[28] Fábio Perez and Ian Ribeiro. 2022. Ignore Previous Prompt: Attack Techniques For Language Models. arXiv:2211.09527. https://arxiv.org/abs/2211.09527

[29] Devin J. Pohly, Stephen McLaughlin, Patrick McDaniel, and Kevin Butler. 2012. Hi-Fi: Collecting High-Fidelity Whole-System Provenance. In *Proceedings of the 28th Annual Computer Security Applications Conference*. Association for Computing Machinery, Orlando, FL, 259–268. doi:10.1145/2420950.2420989

[30] Helio Victor F. Santos, Vitor Costa, Joao Eduardo Montandon, and Marco Tulio Valente. 2025. Decoding the Configuration of AI Coding Agents: Insights from Claude Code Projects. arXiv:2511.09268. https://arxiv.org/abs/2511.09268

[31] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Programmable Privilege Control for LLM Agents. arXiv:2504.11703. https://arxiv.org/abs/2504.11703

[32] KP Singh. 2019. Kernel Runtime Security Instrumentation. Linux Plumbers Conference. https://lpc.events/event/4/contributions/454/

[33] The Linux Kernel Documentation. 2021. LSM BPF Programs. https://www.kernel.org/doc/html/v5.15/bpf/bpf_lsm.html.

[34] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. https://www.langchain.com/blog/the-anatomy-of-an-agent-harness.

[35] Haoyu Wang, Christopher M. Poskitt, and Jun Sun. 2025. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. arXiv:2503.18666. https://arxiv.org/abs/2503.18666

[36] Heng Yin, Dawn Song, Manuel Egele, Christopher Kruegel, and Engin Kirda. 2007. Panorama: Capturing System-wide Information Flow for Malware Detection and Analysis. In *Proceedings of the 14th ACM Conference on Computer and Communications Security*. Association for Computing Machinery, Alexandria, VA, 116–127. doi:10.1145/1315245.1315261

[37] Nickolai Zeldovich, Silas Boyd-Wickizer, Eddie Kohler, and David Mazières. 2006. Making Information Flow Explicit in HiStar. In *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06)*. USENIX Association, 263–278.